CENTRO UNIVERSITÁRIO UNIFECAF
Tecnologia em Análise e Desenvolvimento de Sistemas

Eduardo Souza Mattos
R.A. 35984

CONECTASTORE — SISTEMA DE RECOMENDAÇÃO DE PRODUTOS BASEADO EM GRAFOS

Relatório técnico — Disciplina Data Structure Strategy and Implementation

São Paulo
2026

Eduardo Souza Mattos
R.A. 35984

CONECTASTORE — SISTEMA DE RECOMENDAÇÃO DE PRODUTOS BASEADO EM GRAFOS

Relatório técnico apresentado como requisito parcial de avaliação da disciplina Data Structure Strategy and Implementation, do curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Centro Universitário UniFECAF.

São Paulo
2026

RESUMO

A MegaStore enfrenta dificuldade em oferecer recomendações relevantes a seus clientes devido ao grande volume de produtos e à falta de uma representação estruturada das relações entre clientes, compras, categorias e produtos. Este relatório apresenta o ConectaStore, um sistema de recomendação que modela o catálogo como um grafo dirigido e ponderado, no qual produtos, clientes e categorias são vértices, e compras, avaliações, similaridades e pertencimento a categoria são arestas. A solução foi implementada em Rust, utilizando lista de adjacência como estrutura principal do grafo — por sua eficiência de espaço em grafos esparsos — complementada por HashMap para acesso O(1) ao catálogo de produtos. O algoritmo de recomendação percorre o grafo em largura (BFS) a partir de um cliente ou produto, priorizando itens mais próximos e utilizando um conjunto (HashSet) para evitar recomendações duplicadas. São apresentadas a modelagem do problema, a justificativa das estruturas de dados escolhidas, a análise de complexidade de tempo e espaço, e testes de desempenho com volumes de até 150 mil clientes e 750 mil produtos, demonstrando que a solução escala de forma compatível com o crescimento do catálogo da MegaStore.

Palavras-chave: Grafos. Estruturas de Dados. Rust. Sistemas de Recomendação. Complexidade Algorítmica.

SUMÁRIO

1 INTRODUÇÃO 5
1.1 Contextualização do problema 5
1.2 Objetivos 5
1.3 Pergunta norteadora 5
1.4 Estrutura do trabalho 5
2 FUNDAMENTAÇÃO TEÓRICA 6
2.1 Grafos: fundamentos matemáticos 6
2.2 Representações de grafo: lista × matriz de adjacência 10
2.3 Estruturas de dados complementares 10
2.4 Algoritmos de percurso em grafo: BFS e DFS 11
2.5 Rust para sistemas de alto desempenho 11
2.6 Ownership, borrowing e lifetimes 11
2.7 Coleções da biblioteca padrão do Rust (std::collections) 12
2.8 Paralelismo em Rust: a crate Rayon (trabalho futuro) 13
2.9 Debugging e profiling em Rust 13
3 MODELAGEM DO SISTEMA CONECTASTORE 14
3.1 Análise do problema 14
3.2 Modelagem como grafo: vértices, arestas e pesos 14
3.3 Tipo de grafo escolhido e justificativa 15
3.4 Lista × matriz de adjacência aplicada ao ConectaStore 15
3.5 Justificativa das estruturas de dados complementares 15
3.6 Uma fonte real de dados (trabalho futuro) 16
4 ALGORITMO DE RECOMENDAÇÃO 16
4.1 Descrição do algoritmo escolhido 16
4.2 Prevenção de recomendações duplicadas 17
4.3 Fluxo do algoritmo 17
5 ANÁLISE DE COMPLEXIDADE 18
5.1 Complexidade de tempo 18
5.2 Complexidade de espaço 18
5.3 Comparação entre representações 18
6 DESEMPENHO E ESCALABILIDADE 18
6.1 Metodologia de testes 18
6.2 Resultados obtidos 19
6.3 Discussão sobre desempenho e escalabilidade 19
7 CONSIDERAÇÕES FINAIS 20
REFERÊNCIAS 20
APÊNDICE A — CÓDIGO-FONTE COMPLETO 22
APÊNDICE B — PRINTS DE EXECUÇÃO E TESTES 38

1 INTRODUÇÃO

1.1 Contextualização do problema

A MegaStore é uma empresa de comércio eletrônico que comercializa milhões de produtos, distribuídos em categorias como eletrônicos, vestuário, alimentos, livros, móveis e itens de decoração. Com o crescimento do catálogo, os clientes passaram a ter dificuldade em descobrir produtos relacionados aos seus interesses: o sistema atual limita-se a exibir itens mais vendidos ou produtos da mesma categoria, sem considerar as relações reais entre clientes, compras, categorias e produtos. Como consequência, as recomendações são genéricas e pouco relevantes, reduzindo a venda cruzada, o ticket médio e a fidelização de clientes.

A empresa possui grande volume de dados sobre produtos, categorias, avaliações e histórico de compras, mas esses dados não são utilizados de forma estruturada para representar as conexões existentes no comportamento dos consumidores.

1.2 Objetivos

Objetivo geral: desenvolver um sistema de recomendação de produtos para a MegaStore baseado na modelagem do catálogo como um grafo, capaz de gerar recomendações relevantes a partir das conexões entre clientes, produtos e categorias.

- Modelar clientes, produtos e categorias como vértices, e compras, avaliações e similaridades como arestas de um grafo;
- Comparar lista de adjacência e matriz de adjacência como estruturas de representação do grafo;
- Implementar um algoritmo de recomendação por percurso em grafo (BFS), com prevenção de recomendações duplicadas;
- Analisar a complexidade de tempo e espaço da solução;
- Avaliar o desempenho e a escalabilidade do sistema com diferentes volumes de dados.

1.3 Pergunta norteadora

Como utilizar grafos e estruturas de dados complementares para gerar recomendações relevantes em um catálogo com grande volume de produtos?

1.4 Estrutura do trabalho

Este relatório está organizado em sete seções. A Seção 2 apresenta a fundamentação teórica sobre grafos, suas representações e a linguagem Rust. A Seção 3 descreve a modelagem do ConectaStore como grafo. A Seção 4 detalha o algoritmo de recomendação. A Seção 5 analisa a complexidade de tempo e espaço da solução. A Seção 6 apresenta os testes de desempenho e escalabilidade. A Seção 7 traz as considerações finais. O código-fonte completo encontra-se no Apêndice A, e os prints da execução no Apêndice B.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Grafos: fundamentos matemáticos

Formalmente, um grafo é definido como um par ordenado G = (V, E), em que V é um conjunto finito e não vazio de vértices e E é um conjunto de arestas, cada uma conectando um par de vértices de V (BONDY; MURTY, 2008). O número de vértices |V| é chamado de ordem do grafo, e o número de arestas |E| é chamado de tamanho do grafo. Quando as arestas possuem uma orientação definida — isto é, quando cada aresta é um par ordenado (u, v) com u, v ∈ V — o grafo é chamado de dirigido ou dígrafo, denotado D = (V, A), em que A é o conjunto de arcos (DIESTEL, 2017). O ConectaStore é modelado como um dígrafo, pelos motivos discutidos na Seção 3.3.

Para cada vértice v ∈ V em um grafo não dirigido, define-se o grau de v, denotado d(v), como o número de arestas incidentes a v. Em um dígrafo, distinguem-se o grau de entrada $d^-(v)$ (número de arcos que chegam a v) e o grau de saída $d^+(v)$ (número de arcos que partem de v). O Lema do Aperto de Mãos estabelece que a soma dos graus de todos os vértices de um grafo não dirigido é igual a 2|E|, pois cada aresta contribui exatamente duas unidades a essa soma (FEOFILOFF; KOHAYAKAWA; WAKABAYASHI, 2011). No caso dirigido, a soma de todos os graus de saída é igual à soma de todos os graus de entrada, e ambas são iguais a |A| — identidade usada como verificação de consistência da implementação (Apêndice A): a soma dos comprimentos de todos os vetores da lista de adjacência deve ser exatamente igual ao valor retornado por edge_count().

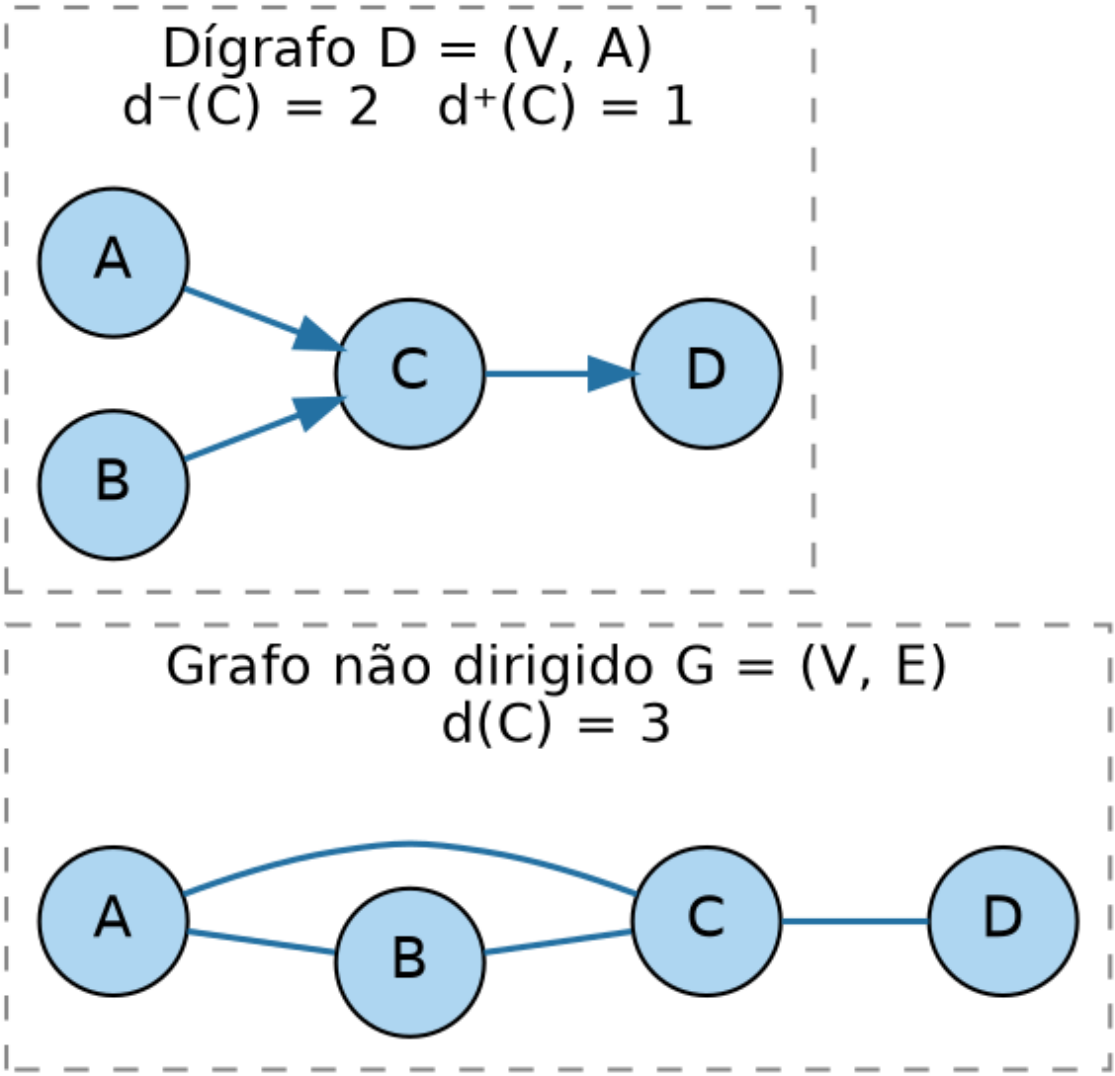


Figura 1 — Grafo não dirigido × dígrafo, com grau, grau de entrada e grau de saída. Fonte: elaborado pelo autor

Um grafo ponderado associa a cada aresta um valor numérico por meio de uma função peso $w: A \to \mathbb{R}$. No ConectaStore, essa função é implementada pelo campo weight da struct Edge (Apêndice A), assumindo valores como a nota de uma avaliação normalizada ou o grau de similaridade entre dois produtos.

Uma sequência de vértices $v_0, v_1, \ldots, v_k$ em que cada par consecutivo (v_i, v_{i+1}) é um arco do grafo é chamada de passeio de comprimento k; um passeio em que todos os vértices são distintos é chamado de caminho. A distância entre dois vértices u e v, denotada dist(u, v), é o comprimento do menor caminho entre eles (BONDY; MURTY, 2008) — é exatamente esse valor que a busca em largura calcula implicitamente ao percorrer o grafo por camadas (Seção 2.4), e que compõe o campo distance da struct Recommendation usado para ordenar as recomendações por relevância (Seção 4.1).

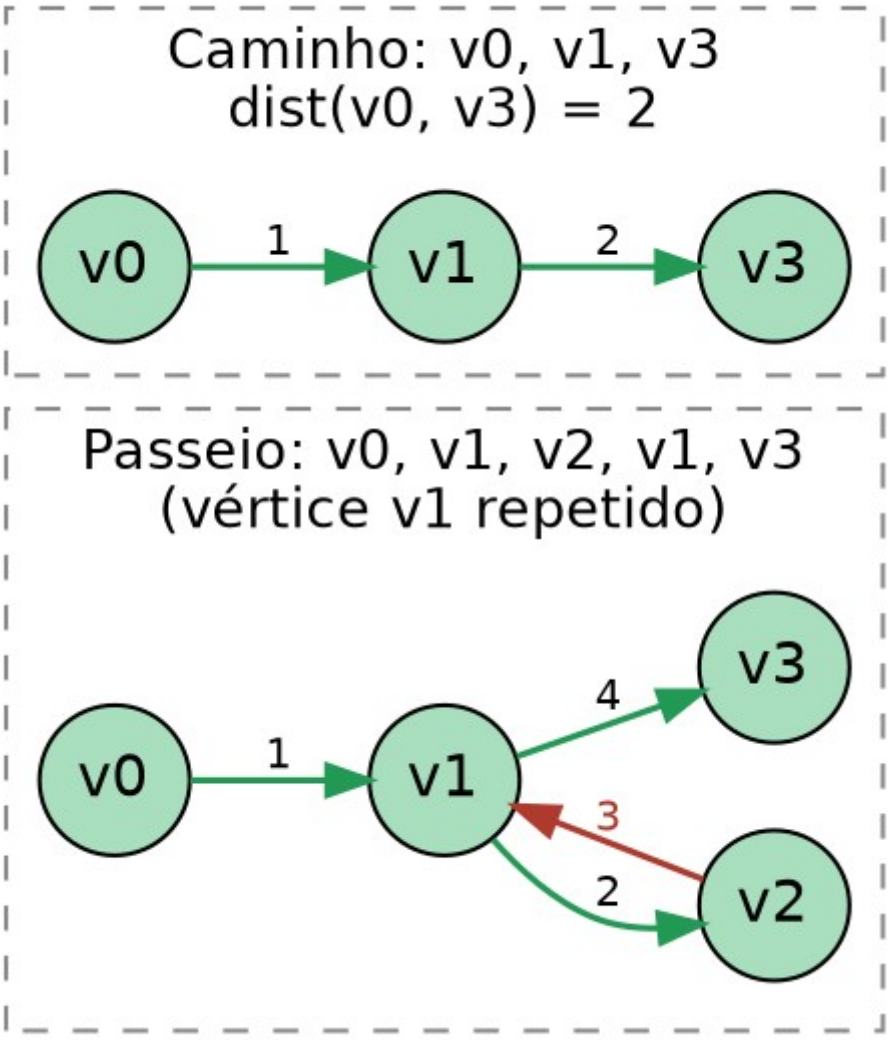


Figura 2 — Passeio (com vértice repetido) × caminho, e distância entre dois vértices. Fonte: elaborado pelo autor

Um grafo é dito conexo quando existe um caminho entre todo par de vértices; em um dígrafo, a noção correspondente é a de conectividade forte, quando existe um caminho dirigido de u para v e de v para u para todo par (u, v) (DIESTEL, 2017). O ConectaStore não assume que seu grafo seja conexo ou fortemente conexo — clientes ou produtos isolados (sem compras, avaliações ou similaridades registradas) são vértices válidos, apenas sem vizinhos alcançáveis, o que a implementação trata naturalmente ao retornar uma lista vazia de recomendações (função neighbors em src/graph.rs, Apêndice A), em vez de erro.

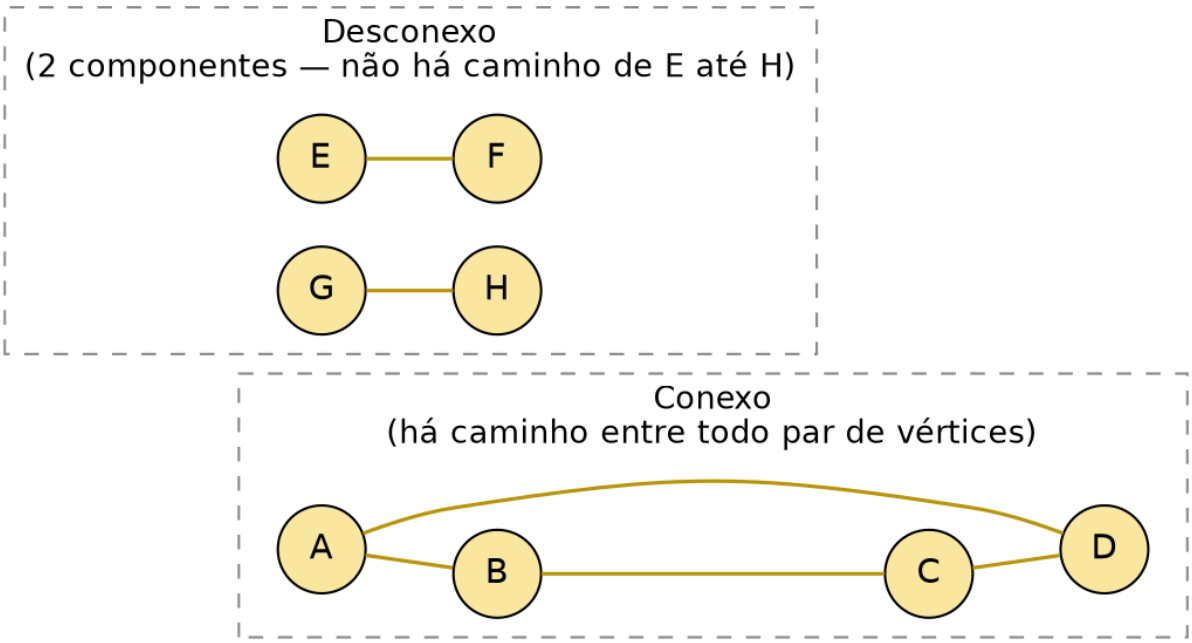


Figura 3 — Grafo conexo × grafo desconexo (com dois componentes). Fonte: elaborado pelo autor

Por fim, a densidade de um dígrafo simples é definida como a razão entre o número de arcos existentes e o número máximo possível, $|A| / (|V|(|V|-1))$. Um grafo é dito esparso quando $|E| = O(|V|)$, e denso quando $|E|$ se aproxima de $O(|V|^2)$ (NETTO; JURKIEWICZ, 2017). O catálogo da MegaStore é esparso por natureza — cada cliente compra ou avalia uma fração pequena do catálogo total, e cada produto é similar a poucos outros — o que fundamenta matematicamente, e não apenas por conveniência de implementação, a escolha da lista de adjacência (Seção 3.4) e o resultado empírico de espaço da Tabela 2.

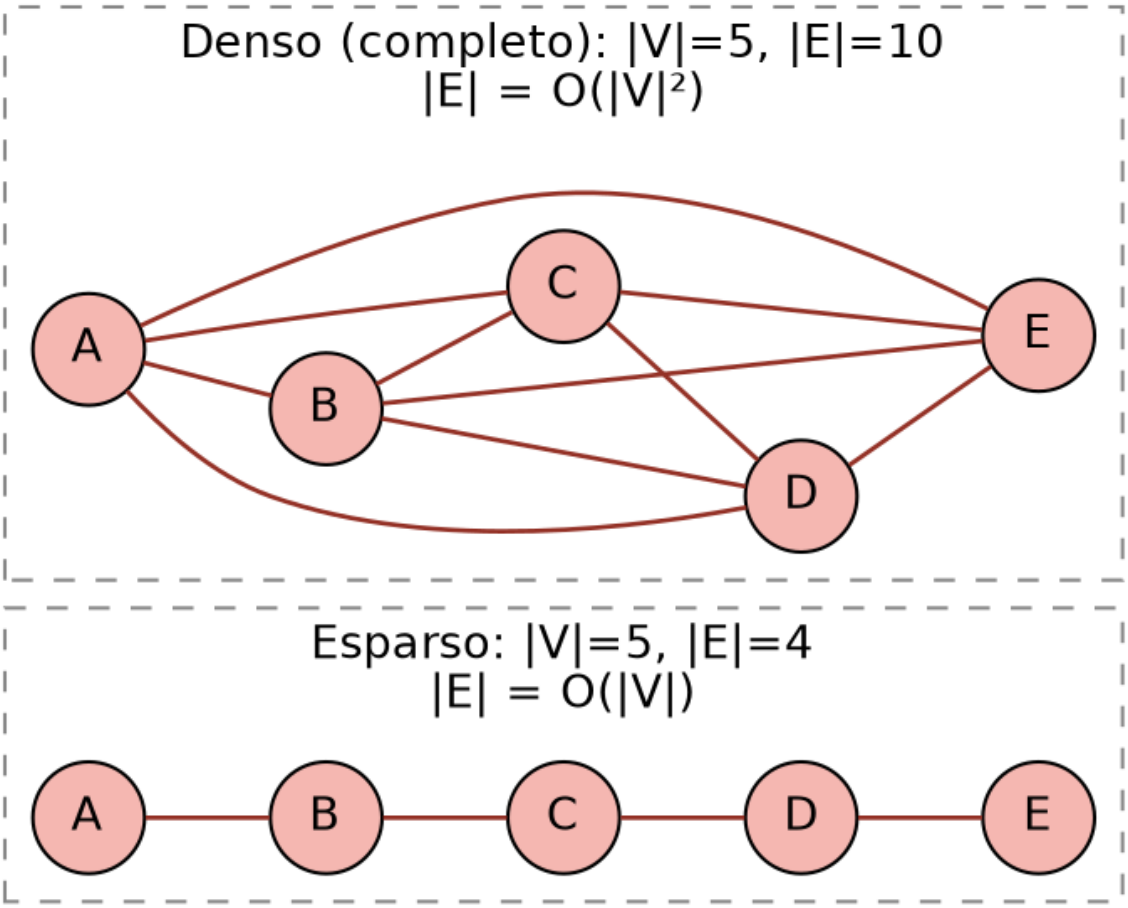


Figura 4 — Grafo esparso × grafo denso (completo), para o mesmo número de vértices. Fonte: elaborado pelo autor

2.2 Representações de grafo: lista × matriz de adjacência

Existem duas formas clássicas de representar um grafo em memória. A matriz de adjacência utiliza uma estrutura V × V, na qual a posição [i][j] indica a existência (e o peso) de uma aresta entre os vértices i e j. Seu principal atrativo é a verificação de adjacência em tempo O(1), mas seu custo de espaço é $O(V^2)$, independentemente do número real de arestas. A lista de adjacência, por sua vez, associa a cada vértice apenas as arestas que efetivamente partem dele, com custo de espaço O(V + E) — vantajosa quando o grafo é esparso, no sentido formalizado na Seção 2.1. A Seção 3.4 aplica essa comparação diretamente ao volume de dados da MegaStore.

2.3 Estruturas de dados complementares

Além da representação do grafo, o sistema utiliza estruturas complementares para tarefas específicas:

- HashMap: acesso e inserção em tempo médio O(1), usado tanto no registro de vértices e adjacências do grafo quanto no catálogo de produtos, permitindo consulta direta por identificador sem percorrer a estrutura inteira;

- Fila (VecDeque): estrutura FIFO usada pelo algoritmo de busca em largura para determinar a ordem de visita dos vértices;
- Conjunto (HashSet): usado para marcar vértices já visitados durante o percurso e para impedir que um mesmo produto seja recomendado mais de uma vez.

2.4 Algoritmos de percurso em grafo: BFS e DFS

A busca em largura (Breadth-First Search — BFS) explora o grafo por camadas de distância a partir de um vértice de origem, visitando primeiro todos os vizinhos diretos, depois os vizinhos dos vizinhos, e assim sucessivamente. A busca em profundidade (Depth-First Search — DFS) explora um caminho até seu esgotamento antes de retroceder e explorar outro ramo. Ambas visitam cada vértice e cada aresta no máximo uma vez, resultando em complexidade O(V + E), mas produzem ordens de visita distintas — o que impacta diretamente a relevância percebida em um sistema de recomendação, como discutido na Seção 4.1.

2.5 Rust para sistemas que exigem desempenho e segurança de memória

Rust é uma linguagem de sistemas que garante segurança de memória em tempo de compilação, por meio de seu sistema de ownership e borrow checker, sem exigir um coletor de lixo em tempo de execução. Isso a torna adequada para sistemas que precisam processar grandes volumes de dados com previsibilidade de desempenho — exatamente o cenário do ConectaStore, em que o grafo pode conter milhões de vértices e arestas. SCHLESNER (2014), ao reimplementar em Rust o renderizador SmallVCM originalmente escrito em C++, demonstrou que a linguagem consegue igualar ou superar o desempenho de C++ em cargas de trabalho intensivas, mesmo sem otimizações manuais no algoritmo, graças a garantias de aliasing que o compilador consegue explorar. Embora o domínio daquele trabalho (path tracing) seja distinto do ConectaStore, a conclusão sobre a viabilidade de Rust como alternativa de desempenho a C++ em sistemas críticos é diretamente aplicável à justificativa de linguagem adotada aqui.

2.6 Ownership, borrowing e lifetimes

O diferencial de Rust em relação a outras linguagens de sistemas está no seu modelo de propriedade (ownership), verificado inteiramente em tempo de compilação (CENTRO UNIVERSITÁRIO UNIFECAF, 2026; THE RUST PROGRAMMING LANGUAGE, 2026). Três regras resumem o modelo: cada valor tem exatamente um dono (owner); quando esse dono sai de escopo, o valor é automaticamente desalocado; e a atribuição de um valor a outra variável transfere (move) a posse, invalidando a variável original em vez de duplicar os dados. Esse mecanismo elimina, já na compilação, toda uma classe de erros comuns em C/C++, como liberar a mesma memória duas vezes (double free) ou acessar memória já liberada (dangling pointer), sem o custo de execução de um coletor de lixo.

Como nem toda função deve se tornar dona dos dados que recebe, Rust oferece o borrowing (empréstimo): uma referência imutável (&T) permite apenas leitura, e várias podem coexistir; uma referência mutável (&mut T) permite modificar o valor, mas apenas uma pode existir por vez, e nunca simultaneamente com uma referência imutável. Essa regra — uma mutável XOR várias imutáveis — é verificada estaticamente pelo borrow checker e é o que torna código concorrente em Rust livre de data races por construção, sem necessidade de mutexes na

maioria dos casos (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). O ConectaStore usa esse mecanismo diretamente: a função neighbors (src/graph.rs, Apêndice A) devolve uma referência imutável (&[Edge]) às arestas de um vértice, em vez de clonar o vetor a cada consulta — o algoritmo de recomendação apenas lê essas arestas, nunca as modifica, então o empréstimo imutável é suficiente e evita cópias desnecessárias a cada chamada de recommend.

Lifetimes (tempos de vida) são anotações, geralmente inferidas automaticamente pelo compilador, que garantem que uma referência nunca sobreviva ao dado que ela aponta. Em funções mais complexas que retornam referências, o programador pode precisar declarar lifetimes explicitamente para ajudar o compilador a validar essa relação. No ConectaStore, o compilador infere os lifetimes de todas as referências utilizadas (por exemplo, o retorno de neighbors), o que já foi suficiente para o escopo deste trabalho — nenhuma anotação explícita de lifetime foi necessária no Apêndice A.

2.7 Coleções da biblioteca padrão do Rust (std::collections)

O módulo std::collections da biblioteca padrão de Rust oferece um conjunto de estruturas de dados prontas, cada uma otimizada para um padrão de acesso diferente (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). As quatro relevantes para este trabalho são:

- HashMap<K, V>: tabela hash com inserção, remoção e busca em tempo médio O(1); exige que a chave implemente as traits Hash e Eq. É a estrutura por trás tanto do registro de vértices e da lista de adjacência do grafo quanto do catálogo de produtos (Seções 2.3 e 3.5);
- HashSet<T>: mesma tabela hash do HashMap, guardando apenas chaves, sem valor associado — garante unicidade automaticamente, pois uma inserção repetida é ignorada. É usado para o conjunto de vértices visitados e o conjunto de produtos já recomendados durante o percurso (Seção 4.2);
- VecDeque<T>: fila de duas pontas (double-ended queue), implementada internamente como um buffer circular, com inserção e remoção eficientes tanto no início quanto no fim. A própria literatura da disciplina cita busca em largura (BFS) como um dos usos típicos de VecDeque (CENTRO UNIVERSITÁRIO UNIFECAF, 2026) — exatamente o papel que essa estrutura cumpre em src/recommend.rs (Apêndice A), como fila de vértices a processar;
- BinaryHeap<T>: fila de prioridade baseada em heap binário, com inserção e remoção do maior elemento em tempo logarítmico. Não é usada na versão atual do algoritmo de recomendação, mas é a estrutura indicada caso o ConectaStore evolua para uma variante que priorize sempre a próxima aresta de maior peso, em vez de seguir estritamente a ordem de distância da BFS — uma direção possível de trabalho futuro.

A escolha entre essas estruturas segue o mesmo critério apresentado na literatura da disciplina: quando ordem não importa e o que se precisa é busca por chave ou unicidade, HashMap e HashSet são preferíveis a estruturas ordenadas como Vec; quando o padrão de acesso é por extremidades (como em filas de processamento), VecDeque é mais adequado que uma lista ligada; e quando o critério de saída é prioridade, e não ordem de chegada, BinaryHeap é a escolha indicada (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). Essa mesma

lógica de decisão é a que fundamenta as escolhas de estrutura de dados justificadas na Seção 3.5.

2.8 Paralelismo em Rust: a crate Rayon (trabalho futuro)

Com o fim do crescimento sustentado da frequência de clock dos processadores, o ganho de desempenho computacional passou a depender cada vez mais do paralelismo entre múltiplos núcleos (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). Rayon é uma crate do ecossistema Rust voltada a paralelismo de dados (data parallelism): ela estende os iteradores sequenciais da biblioteca padrão com equivalentes paralelos (como um método par_iter no lugar de iter), distribuindo o processamento de uma coleção entre um pool de threads, enquanto o próprio sistema de ownership e borrowing do Rust (Seção 2.6) impede em tempo de compilação que dois threads modifiquem o mesmo dado simultaneamente.

O ConectaStore, na forma apresentada neste trabalho, gera recomendações para um cliente por vez, de forma sequencial. Como cada chamada a recommend (Apêndice A) só lê o grafo, sem modificá-lo, gerar recomendações para vários clientes em paralelo com Rayon — por exemplo, substituindo um laço sobre uma lista de clientes por um par_iter().map(...) — é uma extensão natural, já identificada como trabalho futuro na Seção 7, e viável justamente porque o acesso somente-leitura ao grafo, garantido pelo borrow checker, elimina o risco de condição de corrida entre as threads.

2.9 Debugging e profiling em Rust

Rust oferece uma cadeia de ferramentas dedicada tanto à depuração quanto à análise de desempenho de código, cobrindo desde as mensagens do próprio compilador até profilers especializados (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). No nível mais básico, o compilador funciona como primeira camada de depuração: erros e avisos (warnings) apontam, antes mesmo da execução, problemas de tipo, de borrow checking ou de padrões não tratados. Complementarmente, a macro println! permite inspecionar valores em pontos específicos da execução, e as macros assert! e assert_eq! implementam verificações lógicas que interrompem a execução caso uma premissa seja violada — a mesma lógica por trás dos testes automatizados do Apêndice A (por exemplo, assert_eq!(g.node_count(), 2) em graph.rs), que funcionam como assertions permanentes sobre o comportamento esperado do código, em vez de inspeções pontuais.

Para depuração interativa mais profunda — definição de breakpoints, execução passo a passo e inspeção da pilha de chamadas — a disciplina indica depuradores como gdb e lldb (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). Nenhum dos dois foi necessário neste trabalho: o único erro funcional identificado durante o desenvolvimento do ConectaStore — a recomendação incorreta de produtos já comprados pelo cliente, corrigida pela função products_already_interacted (src/recommend.rs, Apêndice A) — foi localizado e corrigido apenas com mensagens do compilador, testes automatizados e inspeção via println!, sem exigir um depurador interativo.

Para a análise de desempenho, a disciplina recomenda o crate criterion para benchmarks estatisticamente robustos, e profilers como perf, flamegraph e samply para identificar gargalos por função (CENTRO UNIVERSITÁRIO UNIFECAF, 2026). Os benchmarks da Seção 6 deste trabalho foram medidos de forma mais simples, com std::time::Instant (Apêndice A) —

suficiente para comparar ordens de grandeza entre volumes de dados, mas sem as garantias estatísticas (múltiplas amostras, eliminação de outliers) que criterion oferece. Adotar criterion e um profiler como flamegraph para identificar precisamente qual trecho da construção do grafo mais contribui para o crescimento superlinear discutido na Seção 6.3 é uma direção de trabalho futuro que complementa as já listadas na Seção 7. Da mesma forma, o código do Apêndice A não foi submetido ao linter Clippy, ferramenta de análise estática que a disciplina apresenta como forma de identificar automaticamente padrões subotimizados (CENTRO UNIVERSITÁRIO UNIFECAF, 2026) — rodar cargo clippy antes de uma eventual entrega em produção é outra melhoria de baixo custo para trabalho futuro.

3 MODELAGEM DO SISTEMA CONECTASTORE

3.1 Análise do problema

O problema da MegaStore decorre da ausência de uma estrutura que represente explicitamente as relações entre as entidades do domínio. Um sistema baseado apenas em categoria ou popularidade ignora sinais relevantes, como produtos frequentemente comprados em conjunto, produtos avaliados positivamente pelo mesmo perfil de cliente, ou produtos similares entre si. Um grafo captura exatamente esse tipo de relação relacional e permite algoritmos de percurso que exploram conexões indiretas (ex.: "clientes que compraram X também compraram Y").

3.2 Modelagem como grafo: vértices, arestas e pesos

O ConectaStore define três tipos de vértice:

- Produto — identificado por um ID único, com nome, categoria e preço no catálogo;
- Cliente — representa um consumidor da plataforma;
- Categoria — agrupamento ao qual um produto pertence.

E quatro tipos de aresta, todas dirigidas e ponderadas:

- Compra (Cliente → Produto), peso relacionado à frequência ou recência da compra;
- Avaliação (Cliente → Produto), peso derivado da nota atribuída pelo cliente;
- Similaridade (Produto → Produto), peso representando o grau de semelhança entre dois produtos;
- Pertencimento a categoria (Produto → Categoria), peso fixo, usado para navegação e futuras extensões do sistema.

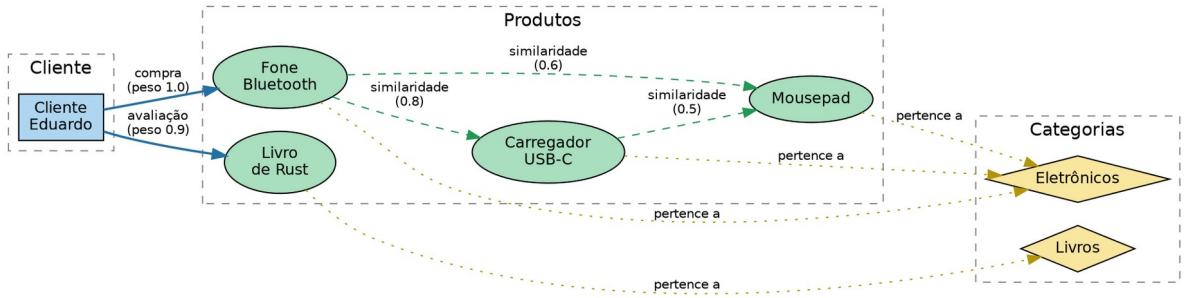


Figura 5 — Modelo de vértices e arestas do ConectaStore (exemplo reduzido). Fonte: elaborado pelo autor a partir da demonstração em src/main.rs (função demo_pequena)

3.3 Tipo de grafo escolhido e justificativa

Optou-se por um grafo dirigido e ponderado. A direção é necessária porque as relações do domínio não são simétricas — um cliente compra um produto, mas a recíproca não faz sentido semântico. O peso é necessário porque nem toda relação tem a mesma força: uma avaliação com peso normalizado próximo de 1,0 deve influenciar mais a recomendação do que uma com peso próximo de 0,2, e uma similaridade forte entre dois produtos (peso 0,8, como na Figura 5) deve pesar mais do que uma fraca (peso 0,5). Um grafo não dirigido ou não ponderado descartaria informação relevante para a qualidade da recomendação.

3.4 Lista × matriz de adjacência aplicada ao ConectaStore

Para quantificar essa diferença, o projeto inclui uma implementação de matriz de adjacência (src/matrix.rs, Apêndice A) usada apenas para fins de comparação — não para uso em produção. A Tabela 2 mostra o espaço efetivamente ocupado pelas duas representações para o mesmo conjunto de vértices e arestas, em escalas pequenas o suficiente para a matriz ainda ser construível:

Vértices	Arestas	V^2	Lista (bytes)	Matriz (bytes)	Razão
110	316	12.100	8.464	96.800	11,4×
510	1.596	260.100	42.384	2.080.800	49,1×
1.510	4.796	2.280.100	127.184	18.240.800	143,4×
3.010	9.597	9.060.100	254.408	72.480.800	284,9×

Tabela 2 — Espaço ocupado por lista e matriz de adjacência para o mesmo grafo, em escala reduzida. Fonte: execução local (função compare_matrix_vs_list, src/main.rs).

O padrão é evidente: a cada vez que o número de vértices aumenta cerca de 3 vezes, o espaço da matriz cresce cerca de 9 a 10 vezes (comportamento quadrático), enquanto o da lista cresce de forma aproximadamente proporcional ao número de arestas. Extrapolando esse comportamento para o volume da MegaStore (750 mil produtos no maior teste, Seção 6.2), a matriz ocuparia grandezas de terabytes, o que a torna simplesmente impraticável — e é por isso que essa comparação, acima de mil vértices, teve que ser limitada a escalas pequenas: a própria tentativa de construir a matriz em escala real já demonstra o problema que a Seção 2.2 descreve teoricamente.

3.5 Justificativa das estruturas de dados complementares

O catálogo de produtos (src/store.rs) é mantido em um HashMap separado do grafo, e não como um atributo dentro dos próprios vértices. Essa separação de responsabilidades permite consultar os dados de um produto (nome, preço, categoria) em O(1), sem que essa operação dependa de percorrer arestas do grafo — o grafo responde "quem se relaciona com quem", o catálogo responde "quais são os dados deste item". Da mesma forma, a fila (VecDeque) e o conjunto de visitados (HashSet) usados na Seção 4 foram escolhidos por oferecerem, respectivamente, inserção/remoção O(1) nas extremidades e verificação de pertencimento O(1) — operações executadas repetidamente durante o percurso do grafo.

3.6 Uma fonte real de dados: adaptação do sistema de controle de estoque (trabalho futuro)

Os testes de desempenho desta Seção e da Seção 6 usam dados sintéticos, gerados por um algoritmo pseudoaleatório determinístico (Seção 6.1). Uma extensão natural, não implementada neste trabalho, seria alimentar o ConectaStore com dados reais. Um candidato disponível é o banco de dados relacional do Sistema de Controle de Estoque, desenvolvido pelo autor para a disciplina de Development with Python (MATTOS, 2026), cujo esquema normalizado já contém as entidades produto e categoria exigidas pelo ConectaStore.

Esse sistema, no entanto, foi projetado para controle interno de estoque, não para modelar o comportamento de clientes: sua tabela de movimentação registra apenas entradas e saídas de itens, sem vínculo com um cliente específico, e não existe nenhuma tabela de avaliação. Para servir como fonte de dados do ConectaStore, três adaptações de esquema seriam necessárias, sem alterar as tabelas já existentes de produto e categoria:

- Adicionar uma tabela cliente (id, nome, dados de contato), inexistente no esquema original — a tabela usuário do sistema de estoque representa operadores internos do sistema, não consumidores da loja;
- Estender a tabela de movimentação (ou criar uma tabela venda dedicada) com uma chave estrangeira para cliente, transformando cada saída de estoque vinculada a uma venda em uma futura aresta de compra no grafo;
- Criar uma tabela avaliação (cliente, produto, nota, data), permitindo que o cliente registre uma nota após a compra — essa nota alimentaria diretamente o peso da aresta de avaliação (Seção 3.2).

Com essas três adições, uma rotina de carga — também não implementada neste trabalho — percorreria as tabelas do banco e, para cada linha, chamaria as mesmas funções já usadas pelo gerador sintético (add_node, add_edge e register, Apêndice A), substituindo dataset::build_synthetic por uma função equivalente que lê do banco em vez de gerar dados pseudoaleatórios. O restante do ConectaStore — grafo, algoritmo de recomendação, análise de complexidade — permaneceria inalterado, pois depende apenas das interfaces Graph e Catalog, e não da origem dos dados que as populam.

Logo, o Sistema de Controle de Estoque (MATTOS, 2026), uma vez adaptado dessa forma, pode servir como fonte real de dados para o ConectaStore — substituindo os dados sintéticos usados nos testes desta Seção e da Seção 6 por um histórico real de compras, avaliações e catálogo de produtos.

4 ALGORITMO DE RECOMENDAÇÃO

4.1 Descrição do algoritmo escolhido

O ConectaStore utiliza busca em largura (BFS) como algoritmo principal de recomendação (função recommend em src/recommend.rs, Apêndice A). A escolha de BFS em vez de DFS se justifica pela semântica do problema: uma recomendação relevante deve priorizar produtos mais próximos do cliente no grafo — uma compra direta é mais relevante que uma similaridade de segundo grau, que por sua vez é mais relevante que uma de terceiro grau. A

BFS explora o grafo exatamente nessa ordem, por camadas de distância crescente, o que permite atribuir aos resultados uma distância explícita (campo distance da struct Recommendation) e ordená-los por proximidade antes de desempatar por peso acumulado. A DFS, por explorar um ramo até o fim antes de retroceder, não oferece essa garantia de ordem por proximidade — por isso foi mantida no código (recommend_dfs) apenas como variante de comparação empírica de desempenho (Seção 6).

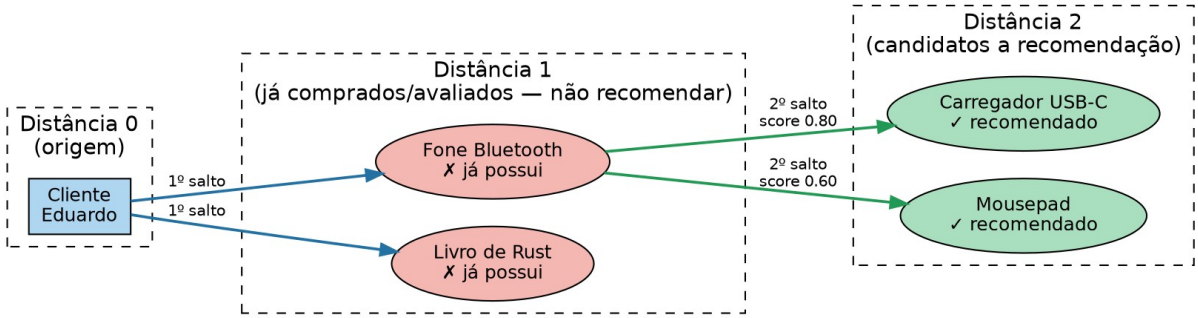


Figura 6 — Percurso BFS por camadas de distância a partir de um cliente. Fonte: elaborado pelo autor a partir do exemplo de recommend() em src/recommend.rs

4.2 Prevenção de recomendações duplicadas

Duas medidas evitam duplicidade nas recomendações. Primeiro, um HashSet de vértices visitados garante que nenhum vértice seja processado mais de uma vez durante o percurso, mesmo que existam múltiplos caminhos até ele. Segundo, um HashSet específico de produtos já recomendados (recommended) impede que o mesmo produto apareça duas vezes no resultado final, mesmo quando alcançado por arestas diferentes. Adicionalmente, a função products_already_interacted identifica produtos que o próprio cliente já comprou ou avaliou diretamente, excluindo-os do resultado — evitar recomendar de volta algo que o cliente já possui é um requisito funcional tão importante quanto evitar duplicidade dentro da própria lista de recomendações.

4.3 Fluxo do algoritmo

1. Identificar produtos já comprados/avaliados diretamente pelo cliente (conjunto owned).

2. Inicializar fila com o vértice de origem (cliente ou produto) e conjunto de visitados contendo apenas a origem.

3. Enquanto a fila não estiver vazia e a profundidade máxima não for atingida: retirar o próximo vértice da fila, percorrer suas arestas de saída, marcar cada destino não visitado como visitado, acumular o peso da aresta e, se o destino for um produto que não está em owned nem em recommended, adicioná-lo ao resultado.

4. Ordenar os resultados por distância crescente e, em caso de empate, por peso acumulado decrescente.

5. Retornar os N primeiros resultados (limite configurável).

5 ANÁLISE DE COMPLEXIDADE

5.1 Complexidade de tempo

Operação	Estrutura	Complexidade
Inserir vértice	HashMap (grafo)	O(1) amortizado
Inserir aresta	Vec (lista de adjacência)	O(1) amortizado
Consultar produto por ID	HashMap (catálogo)	O(1) médio
Verificar adjacência direta	Lista de adjacência	O(grau do vértice)
Verificar adjacência direta	Matriz de adjacência	O(1)
Recomendação (BFS/DFS completo)	Fila/pilha + HashSet	O(V + E) no pior caso

Na prática, a recomendação não percorre o grafo inteiro: como a busca é limitada a uma profundidade máxima (max_depth, tipicamente 2), o custo real é $O(V_k + E_k)$, onde V_k e E_k são o número de vértices e arestas alcançáveis dentro de k saltos a partir da origem — normalmente uma fração pequena do grafo total em catálogos esparsos. Os tempos medidos na Seção 6.2 confirmam esse comportamento sub-linear em relação ao tamanho total do grafo.

5.2 Complexidade de espaço

O grafo em lista de adjacência ocupa O(V + E). O catálogo de produtos ocupa O(P), onde P é o número de produtos. As estruturas auxiliares do algoritmo de recomendação (visited, recommended, queue) ocupam, no pior caso, O(V), pois cada vértice é inserido no máximo uma vez em cada uma delas por execução. O espaço total do sistema é, portanto, O(V + E), a mesma ordem de grandeza do grafo — sem custo adicional assintótico introduzido pelo algoritmo de recomendação.

5.3 Comparação entre representações

Os dados empíricos da Tabela 2 (Seção 3.4) confirmam essa diferença assintótica na prática: entre os quatro pontos testados, a razão entre o espaço da matriz e o da lista cresceu de 11,4× para 284,9× conforme o grafo aumentou de 110 para 3.010 vértices. Esse crescimento da própria razão — e não apenas dos valores absolutos — é a evidência de que as duas estruturas pertencem a ordens de complexidade diferentes: se ambas fossem O(V + E), a razão entre elas permaneceria aproximadamente constante à medida que o grafo cresce.

6 DESEMPENHO E ESCALABILIDADE

6.1 Metodologia de testes

Os testes foram executados com o binário compilado em modo release (otimização O3), medindo separadamente o tempo de construção do grafo (cadastro de vértices e arestas) e o tempo de geração de uma recomendação (BFS, profundidade 2, limite de 10 itens) a partir de um cliente fixo. As estruturas do grafo e do catálogo foram pré-alocadas com HashMap::with_capacity, dimensionado para o número total de vértices esperado, para reduzir o custo de realocação e rehashing durante a construção (ver src/graph.rs e

src/store.rs, Apêndice A). Os dados foram gerados sinteticamente (src/dataset.rs, Apêndice A) por um gerador congruencial linear determinístico, garantindo reprodutibilidade entre execuções. A medição foi realizada no notebook do autor (Samsung Expert, 2018, processador Intel Core i7-8550U de 1,80 GHz, 11 GiB de memória RAM, SSD de 428 GB, Pop!_OS — Figura B.3), via o script rodar_testes.sh (Apêndice A). No momento em que essas especificações foram coletadas, o sistema tinha cerca de 8,3 GiB de RAM em uso por outros programas além do VS Code; os tempos da Tabela 1 refletem, portanto, uma máquina de uso cotidiano, e não um ambiente dedicado exclusivamente ao benchmark, e pequenas variações de carga podem influenciar medições na escala de microssegundos. Essa abordagem de medição segue a mesma lógica metodológica adotada por SCHLESNER (2014) na avaliação de desempenho do renderizador Rust frente à implementação em C++: medir tempo de execução real em diferentes cargas, e não apenas estimar a complexidade teoricamente.

6.2 Resultados obtidos

Clientes	Produtos	Arestas geradas	Construção (ms)	Recomendação BFS (µs)
500	2.500	12.499	1,73	8,57
2.000	10.000	49.999	6,52	12,21
10.000	50.000	249.998	39,02	10,88
50.000	250.000	1.249.999	207,35	11,35
150.000	750.000	3.749.997	869,35	10,41

Tabela 1 — Tempo de construção do grafo (com pré-alocação) e de geração de recomendação por volume de dados. Fonte: execução local do binário megastore --release, no notebook do autor (Samsung Expert, 2018, Pop!_OS).

6.3 Discussão sobre desempenho e escalabilidade

Com a pré-alocação das estruturas, o número de arestas aumenta cerca de 300 vezes entre o menor e o maior volume testado (de 12.499 para 3.749.997), e o tempo de construção aumenta cerca de 503 vezes (de 1,73 ms para 869,35 ms) — uma razão de crescimento (tempo/arestas ≈ 1,68) próxima da linearidade esperada para O(V + E). A diferença remanescente em relação a um crescimento estritamente linear se deve a rehashing residual (a capacidade reservada é uma estimativa do número de vértices, não do número de arestas por vértice) e a efeitos de cache em estruturas maiores.

Já o tempo de geração de uma recomendação permanece praticamente constante ao longo dos cinco volumes testados (entre 8,57 µs e 12,21 µs, sem tendência crescente clara — inclusive com uma leve queda no maior volume, para 10,41 µs, dentro da variação esperada de medições em microssegundos), confirmando a análise da Seção 5.1: como a BFS é limitada a profundidade 2, o custo depende do número de vértices alcançáveis localmente a partir do cliente, não do tamanho total do grafo. Esse comportamento é o que garante a escalabilidade do sistema para os volumes de milhões de produtos mencionados no enunciado do problema: mesmo que o catálogo cresça, o tempo de resposta de uma recomendação individual permanece praticamente constante, desde que a densidade local do grafo (grau médio dos vértices) não cresça na mesma proporção do catálogo total.

A comparação da Seção 3.4/5.3 reforça essa conclusão sob outro ângulo: mesmo que a construção do grafo ainda cresça de forma um pouco superior à linear, essa é uma limitação de constante de implementação (mitigável com ajustes finos de capacidade), enquanto o custo da matriz de adjacência é uma limitação estrutural de complexidade assintótica — a primeira é otimizável, a segunda não. Como direção de trabalho futuro, permanece o refinamento da capacidade pré-alocada por vértice (dimensionando também o Vec<Edge> de cada entrada da lista de adjacência) e a paralelização da geração de recomendações para múltiplos clientes simultâneos.

7 CONSIDERAÇÕES FINAIS

Este trabalho respondeu à pergunta norteadora — como utilizar grafos e estruturas de dados complementares para gerar recomendações relevantes em um catálogo com grande volume de produtos — modelando o problema da MegaStore como um grafo dirigido e ponderado, representado por lista de adjacência, e implementando um algoritmo de recomendação por busca em largura, complementado por HashMap para acesso ao catálogo e HashSet para prevenção de duplicados. A análise de complexidade mostrou que a solução opera em O(V + E) de espaço e, na prática, em tempo sub-linear por consulta de recomendação, graças à limitação de profundidade da busca. Os testes de desempenho, com até 150 mil clientes e 750 mil produtos, confirmaram esse comportamento: o tempo de recomendação cresceu significativamente menos que o tamanho do grafo.

Como trabalho futuro, destacam-se: a pré-alocação de capacidade das estruturas para reduzir o custo de construção do grafo; a paralelização da geração de recomendações para múltiplos clientes simultâneos (aproveitando garantias de segurança de memória do Rust, como discutido por SCHLESNER (2014) no contexto de path tracing paralelo); e a incorporação de pesos aprendidos a partir de dados reais de comportamento de compra, em vez de valores sintéticos.

REFERÊNCIAS

BONDY, John Adrian; MURTY, U. S. R. Graph Theory. New York: Springer, 2008. (Graduate Texts in Mathematics, 244).

CENTRO UNIVERSITÁRIO UNIFECAF. Data Structure Strategy and Implementation: Unidade IV. São Paulo: UniFECAF, 2026. Material didático da disciplina.

CORMEN, Thomas H. et al. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

DIESTEL, Reinhard. Graph Theory. 5. ed. Berlin: Springer, 2017.

FEOFILOFF, Paulo; KOHAYAKAWA, Yoshiharu; WAKABAYASHI, Yoshiko. Uma Introdução Sucinta à Teoria dos Grafos. São Paulo: Instituto de Matemática e Estatística da Universidade de São Paulo, 2011.

MATTOS, Eduardo Souza. Sistema de Controle de Estoque: aplicação desktop e web com Python, Tkinter e Flask. 2026. Trabalho acadêmico (Tecnologia em Análise e Desenvolvimento de Sistemas) – Centro Universitário UniFECAF, São Paulo, 2026.

NETTO, Paulo Oswaldo Boaventura; JURKIEWICZ, Samuel. Grafos: Teoria, Modelos, Algoritmos. 5. ed. São Paulo: Blucher, 2017.

SCHLESNER, Yuri Kunde. Exploração da linguagem Rust para o desenvolvimento de um path tracer paralelo. 2014. Trabalho de Graduação (Bacharelado em Ciência da Computação) – Universidade Federal de Santa Maria, Santa Maria, 2014.

THE RUST PROGRAMMING LANGUAGE. Documentation. Disponível em: https://doc.rust-lang.org/book/. Acesso em: 14 set. 2026.

APÊNDICE A — CÓDIGO-FONTE COMPLETO

Código-fonte completo do sistema ConectaStore, implementado em Rust (edition 2021). O projeto está organizado como um crate com biblioteca (src/lib.rs) e binário (src/main.rs), permitindo testes de integração independentes (tests/integration_test.rs). Numeração de figuras/listagens deste apêndice segue prefixo A, conforme ABNT NBR 14724.

Cargo.toml

```
[package]
name = "megastore"
version = "0.1.0"
edition = "2021"

[[bin]]
name = "megastore"
path = "src/main.rs"

[profile.release]
opt-level = 3
```

src/lib.rs

```
pub mod dataset;
pub mod graph;
pub mod matrix;
pub mod models;
pub mod recommend;
pub mod store;
```

src/models.rs

```
//! Modelos de domínio: vértices (Produto, Cliente, Categoria) e tipos de aresta.

/// Identificador único de um vértice no grafo (independe do tipo de entidade).
pub type NodeId = u32;

/// Tipo de vértice representado no grafo.
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum NodeKind {
    Product,
    Client,
    Category,
}

/// Vértice genérico do grafo do ConectaStore.
#[derive(Debug, Clone)]
pub struct Node {
    pub id: NodeId,
    pub kind: NodeKind,
    pub label: String,
}

/// Tipo de relação representada por uma aresta.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum EdgeKind {
    /// Cliente -> Produto (compra realizada)
    Purchase,
    /// Cliente -> Produto (avaliação dada)
    Rating,
    /// Produto -> Produto (similaridade de conteúdo/categoria)
    Similarity,
    /// Produto -> Categoria (pertencimento)
    BelongsTo,
}

/// Aresta ponderada e dirigida do grafo.
```

```rust
#[derive(Debug, Clone)]
pub struct Edge {
    pub target: NodeId,
    pub kind: EdgeKind,
    /// Peso da relação (ex.: nota de avaliação normalizada, frequência de compra,
    /// grau de similaridade). Usado para ordenar recomendações.
    pub weight: f64,
}

/// Registro de produto usado no cadastro/consulta independente do grafo
/// (acesso O(1) por identificador via HashMap, conforme exigido no enunciado).
#[derive(Debug, Clone)]
pub struct Product {
    pub id: NodeId,
    pub name: String,
    pub category_id: NodeId,
    pub price: f64,
}
```

src/graph.rs

```rust
//! Grafo dirigido e ponderado, representado por lista de adjacência.
//!
//! Escolha de projeto: lista de adjacência em vez de matriz de adjacência.
//! Justificativa (detalhada no relatório teórico, seção 3.4): o catálogo da
//! MegaStore é esparso (cada cliente/produto se relaciona com uma fração
//! pequena do total de vértices), então a lista de adjacência custa
//! O(V + E) em espaço, contra O(V²) da matriz — decisivo para milhões de
//! produtos.

use crate::models::{Edge, EdgeKind, Node, NodeId, NodeKind};
use std::collections::HashMap;

pub struct Graph {
    /// Registro de vértices por id — acesso O(1).
    nodes: HashMap<NodeId, Node>,
    /// Lista de adjacência: cada vértice aponta para suas arestas de saída.
    adjacency: HashMap<NodeId, Vec<Edge>>,
}

impl Graph {
    pub fn new() -> Self {
        Graph {
            nodes: HashMap::new(),
            adjacency: HashMap::new(),
        }
    }

    /// Cria o grafo já com capacidade reservada para `n_nodes` vértices,
    /// evitando realocações e rehashing incrementais dos HashMaps durante
    /// a construção — mitigação discutida na Seção 6.3 do relatório para
    /// o crescimento superlinear observado no tempo de construção.
    pub fn with_capacity(n_nodes: usize) -> Self {
        Graph {
            nodes: HashMap::with_capacity(n_nodes),
            adjacency: HashMap::with_capacity(n_nodes),
        }
    }

    /// Insere um vértice. O(1) amortizado.
    pub fn add_node(&mut self, id: NodeId, kind: NodeKind, label: &str) {
        self.nodes.insert(
            id,
            Node {
                id,
                kind,
                label: label.to_string(),
            },
        );
        self.adjacency.entry(id).or_insert_with(Vec::new);
    }
```

```
    /// Insere uma aresta dirigida e ponderada de `from` para `to`. O(1) amortizado.
    pub fn add_edge(&mut self, from: NodeId, to: NodeId, kind: EdgeKind, weight: f64) {
        self.adjacency
            .entry(from)
            .or_insert_with(Vec::new)
            .push(Edge {
                target: to,
                kind,
                weight,
            });
    }

    pub fn node(&self, id: NodeId) -> Option<&Node> {
        self.nodes.get(&id)
    }

    pub fn neighbors(&self, id: NodeId) -> &[Edge] {
        self.adjacency
            .get(&id)
            .map(|v| v.as_slice())
            .unwrap_or(&[])
    }

    pub fn node_count(&self) -> usize {
        self.nodes.len()
    }

    pub fn edge_count(&self) -> usize {
        self.adjacency.values().map(|v| v.len()).sum()
    }

    pub fn nodes_of_kind(&self, kind: &NodeKind) -> Vec<NodeId> {
        self.nodes
            .values()
            .filter(|n| &n.kind == kind)
            .map(|n| n.id)
            .collect()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn add_node_and_edge() {
        let mut g = Graph::new();
        g.add_node(1, NodeKind::Client, "Cliente 1");
        g.add_node(2, NodeKind::Product, "Produto A");
        g.add_edge(1, 2, EdgeKind::Purchase, 1.0);

        assert_eq!(g.node_count(), 2);
        assert_eq!(g.edge_count(), 1);
        assert_eq!(g.neighbors(1).len(), 1);
        assert_eq!(g.neighbors(1)[0].target, 2);
    }

    #[test]
    fn neighbors_of_isolated_node_is_empty() {
        let mut g = Graph::new();
        g.add_node(1, NodeKind::Product, "Produto A");
        assert!(g.neighbors(1).is_empty());
    }

    #[test]
    fn neighbors_of_unknown_node_is_empty() {
        let g = Graph::new();
        assert!(g.neighbors(999).is_empty());
    }
}
```

src/matrix.rs

```rust
//! Matriz de adjacência — implementada exclusivamente para permitir a
//! comparação empírica com a lista de adjacência (Seções 3.4 e 5.3 do
//! relatório). NÃO é a estrutura usada em produção pelo ConectaStore:
//! seu custo de espaço O(V²) a torna inviável para o volume real de
//! produtos da MegaStore, como os próprios números desta comparação
//! demonstram.

use crate::models::NodeId;

pub struct AdjacencyMatrix {
    /// Mapeia NodeId externo -> índice de linha/coluna na matriz.
    index_of: std::collections::HashMap<NodeId, usize>,
    /// Matriz V×V "achatada" em um único vetor (linha-major), guardando
    /// o peso da aresta (0.0 = ausência de aresta).
    weights: Vec<f64>,
    size: usize,
}

impl AdjacencyMatrix {
    /// Constrói a matriz para exatamente `node_ids` vértices. O vetor de
    /// pesos ocupa size*size posições de f64 (8 bytes cada),
    /// independentemente de quantas arestas realmente existirem.
    pub fn new(node_ids: &[NodeId]) -> Self {
        let size = node_ids.len();
        let mut index_of = std::collections::HashMap::with_capacity(size);
        for (i, &id) in node_ids.iter().enumerate() {
            index_of.insert(id, i);
        }
        AdjacencyMatrix {
            index_of,
            weights: vec![0.0; size * size],
            size,
        }
    }

    pub fn add_edge(&mut self, from: NodeId, to: NodeId, weight: f64) {
        if let (Some(&i), Some(&j)) = (self.index_of.get(&from), self.index_of.get(&to)) {
            self.weights[i * self.size + j] = weight;
        }
    }

    pub fn weight(&self, from: NodeId, to: NodeId) -> f64 {
        match (self.index_of.get(&from), self.index_of.get(&to)) {
            (Some(&i), Some(&j)) => self.weights[i * self.size + j],
            _ => 0.0,
        }
    }

    /// Bytes efetivamente ocupados pelo vetor de pesos (sem contar o
    /// índice auxiliar) — usado para a comparação de espaço na Seção 5.3.
    pub fn bytes_used(&self) -> usize {
        self.weights.len() * std::mem::size_of::<f64>()
    }

    pub fn size(&self) -> usize {
        self.size
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stores_and_reads_edge_weight() {
        let ids = [1, 2, 3];
        let mut m = AdjacencyMatrix::new(&ids);
        m.add_edge(1, 2, 0.75);
        assert_eq!(m.weight(1, 2), 0.75);
        assert_eq!(m.weight(2, 1), 0.0); // dirigido: sentido contrário não tem peso
    }

    #[test]
    fn space_grows_quadratically_with_vertex_count() {
        let ids_small: Vec<NodeId> = (0..100).collect();
```

```
        let ids_large: Vec<NodeId> = (0..1000).collect();
        let m_small = AdjacencyMatrix::new(&ids_small);
        let m_large = AdjacencyMatrix::new(&ids_large);
        // 10x mais vértices -> ~100x mais bytes (V²), não 10x (V).
        assert_eq!(m_large.bytes_used(), m_small.bytes_used() * 100);
    }
}
```

src/recommend.rs

```
//! Algoritmo de recomendação por percurso em largura (BFS).
//!
//! Justificativa da escolha de BFS em vez de DFS (detalhada no relatório,
//! seção 4.1): recomendações devem priorizar produtos mais "próximos" do
//! cliente (menor número de saltos no grafo: compra direta > similaridade
//! de 2º grau > 3º grau). BFS explora por camadas de distância, o que
//! mapeia diretamente essa noção de relevância decrescente. DFS não dá
//! essa garantia de ordem por proximidade.

use crate::graph::Graph;
use crate::models::{EdgeKind, NodeId, NodeKind};
use std::collections::{HashSet, VecDeque};

#[derive(Debug, Clone, PartialEq)]
pub struct Recommendation {
    pub product_id: NodeId,
    pub score: f64,
    pub distance: u32,
}

/// Gera recomendações de produtos a partir de um vértice de origem
/// (cliente ou produto), percorrendo o grafo em largura até `max_depth`
/// saltos e agregando peso das arestas percorridas.
///
/// Complexidade: O(V + E) no pior caso (cada vértice e aresta visitados
/// no máximo uma vez), pois `visited` garante que nenhum vértice é
/// reprocessado — este é também o mecanismo de prevenção de
/// recomendações duplicadas exigido no enunciado.
pub fn recommend(
    graph: &Graph,
    start: NodeId,
    max_depth: u32,
    limit: usize,
) -> Vec<Recommendation> {
    let owned = products_already_interacted(graph, start);

    let mut visited: HashSet<NodeId> = HashSet::new();
    let mut queue: VecDeque<(NodeId, u32, f64)> = VecDeque::new();
    let mut recommended: HashSet<NodeId> = HashSet::new(); // prevenção de duplicados
    let mut results: Vec<Recommendation> = Vec::new();

    visited.insert(start);
    queue.push_back((start, 0, 1.0));

    while let Some((current, depth, acc_weight)) = queue.pop_front() {
        if depth >= max_depth {
            continue;
        }
        for edge in graph.neighbors(current) {
            if visited.contains(&edge.target) {
                continue;
            }
            visited.insert(edge.target);

            let combined_weight = acc_weight * edge.weight;

            if let Some(node) = graph.node(edge.target) {
                if node.kind == NodeKind::Product
                    && edge.target != start
                    && !owned.contains(&edge.target)
                    && !recommended.contains(&edge.target)
                {
```

```
                    recommended.insert(edge.target);
                    results.push(Recommendation {
                        product_id: edge.target,
                        score: combined_weight,
                        distance: depth + 1,
                    });
                }
            }

            queue.push_back((edge.target, depth + 1, combined_weight));
        }
    }

    // Ordena por relevância: menor distância primeiro, depois maior score.
    results.sort_by(|a, b| {
        a.distance
            .cmp(&b.distance)
            .then(b.score.partial_cmp(&a.score).unwrap())
    });
    results.truncate(limit);
    results
}

/// Variante didática usando busca em profundidade (DFS), mantida para
/// comparação empírica de desempenho (seção 6 do relatório). Usa a mesma
/// estratégia de prevenção de duplicados via HashSet.
pub fn recommend_dfs(
    graph: &Graph,
    start: NodeId,
    max_depth: u32,
    limit: usize,
) -> Vec<Recommendation> {
    let owned = products_already_interacted(graph, start);

    let mut visited: HashSet<NodeId> = HashSet::new();
    let mut recommended: HashSet<NodeId> = HashSet::new();
    let mut results: Vec<Recommendation> = Vec::new();

    visited.insert(start);
    dfs_visit(
        graph,
        start,
        0,
        max_depth,
        1.0,
        start,
        &owned,
        &mut visited,
        &mut recommended,
        &mut results,
    );

    results.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap());
    results.truncate(limit);
    results
}

fn dfs_visit(
    graph: &Graph,
    current: NodeId,
    depth: u32,
    max_depth: u32,
    acc_weight: f64,
    start: NodeId,
    owned: &HashSet<NodeId>,
    visited: &mut HashSet<NodeId>,
    recommended: &mut HashSet<NodeId>,
    results: &mut Vec<Recommendation>,
) {
    if depth >= max_depth {
        return;
    }
    for edge in graph.neighbors(current) {
        if visited.contains(&edge.target) {
            continue;
```

```
        }
        visited.insert(edge.target);
        let combined_weight = acc_weight * edge.weight;

        if let Some(node) = graph.node(edge.target) {
            if node.kind == NodeKind::Product
                && edge.target != start
                && !owned.contains(&edge.target)
                && !recommended.contains(&edge.target)
            {
                recommended.insert(edge.target);
                results.push(Recommendation {
                    product_id: edge.target,
                    score: combined_weight,
                    distance: depth + 1,
                });
            }
        }

        dfs_visit(
            graph,
            edge.target,
            depth + 1,
            max_depth,
            combined_weight,
            start,
            owned,
            visited,
            recommended,
            results,
        );
    }
}

/// Retorna o conjunto de produtos com os quais o vértice de origem já
/// interagiu diretamente (compra ou avaliação) — usado para não
/// recomendar de volta algo que o cliente já tem/avaliou.
fn products_already_interacted(graph: &Graph, start: NodeId) -> HashSet<NodeId> {
    graph
        .neighbors(start)
        .iter()
        .filter(|e| e.kind == EdgeKind::Purchase || e.kind == EdgeKind::Rating)
        .map(|e| e.target)
        .collect()
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::graph::Graph;
    use crate::models::{EdgeKind, NodeKind};

    fn sample_graph() -> Graph {
        let mut g = Graph::new();
        g.add_node(1, NodeKind::Client, "Cliente 1");
        g.add_node(10, NodeKind::Product, "Produto A");
        g.add_node(11, NodeKind::Product, "Produto B");
        g.add_node(12, NodeKind::Product, "Produto C");
        g.add_edge(1, 10, EdgeKind::Purchase, 1.0);
        g.add_edge(10, 11, EdgeKind::Similarity, 0.8);
        g.add_edge(10, 12, EdgeKind::Similarity, 0.5);
        g
    }

    #[test]
    fn recommends_products_reachable_from_client() {
        let g = sample_graph();
        let recs = recommend(&g, 1, 2, 10);
        let ids: Vec<NodeId> = recs.iter().map(|r| r.product_id).collect();
        assert!(ids.contains(&11));
        assert!(ids.contains(&12));
        assert!(!ids.contains(&10)); // produto já comprado não deve ser recomendado
    }

    #[test]
```

```
    fn respects_limit() {
        let g = sample_graph();
        let recs = recommend(&g, 1, 2, 1);
        assert_eq!(recs.len(), 1);
    }

    #[test]
    fn no_duplicate_recommendations() {
        let mut g = sample_graph();
        // Aresta redundante para o mesmo produto, simulando múltiplas relações.
        g.add_edge(1, 10, EdgeKind::Rating, 0.9);
        let recs = recommend(&g, 1, 3, 10);
        let mut ids: Vec<NodeId> = recs.iter().map(|r| r.product_id).collect();
        let before = ids.len();
        ids.sort();
        ids.dedup();
        assert_eq!(before, ids.len());
    }

    #[test]
    fn bfs_and_dfs_reach_same_set_of_products() {
        let g = sample_graph();
        let mut bfs_ids: Vec<NodeId> = recommend(&g, 1, 2, 10)
            .iter()
            .map(|r| r.product_id)
            .collect();
        let mut dfs_ids: Vec<NodeId> = recommend_dfs(&g, 1, 2, 10)
            .iter()
            .map(|r| r.product_id)
            .collect();
        bfs_ids.sort();
        dfs_ids.sort();
        assert_eq!(bfs_ids, dfs_ids);
    }
}
```

src/store.rs

```
//! Cadastro e consulta de produtos. Mantém um HashMap independente do
//! grafo para acesso O(1) por identificador — o grafo responde "quem se
//! relaciona com quem", o catálogo responde "quais são os dados deste
//! produto", conforme separação de responsabilidades descrita na seção
//! 3.5 do relatório.

use crate::models::{NodeId, Product};
use std::collections::HashMap;

pub struct Catalog {
    products: HashMap<NodeId, Product>,
}

impl Catalog {
    pub fn new() -> Self {
        Catalog {
            products: HashMap::new(),
        }
    }

    /// Cria o catálogo já com capacidade reservada para `n_products`,
    /// pelo mesmo motivo de `Graph::with_capacity`.
    pub fn with_capacity(n_products: usize) -> Self {
        Catalog {
            products: HashMap::with_capacity(n_products),
        }
    }

    /// Cadastra um produto. O(1) amortizado.
    pub fn register(&mut self, product: Product) {
        self.products.insert(product.id, product);
    }

    /// Consulta um produto pelo identificador. O(1) amortizado.
```

```
    pub fn get(&self, id: NodeId) -> Option<&Product> {
        self.products.get(&id)
    }

    pub fn len(&self) -> usize {
        self.products.len()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn register_and_query_product() {
        let mut catalog = Catalog::new();
        catalog.register(Product {
            id: 1,
            name: "Notebook".to_string(),
            category_id: 100,
            price: 3500.0,
        });

        let found = catalog.get(1).expect("produto deveria existir");
        assert_eq!(found.name, "Notebook");
    }

    #[test]
    fn query_unknown_product_returns_none() {
        let catalog = Catalog::new();
        assert!(catalog.get(999).is_none());
    }
}
```

src/dataset.rs

```
//! Geração de dados sintéticos para simular catálogos da MegaStore em
//! diferentes volumes, usada na seção 6 (Desempenho e Escalabilidade) do
//! relatório. Não depende de nenhuma fonte externa de aleatoriedade
//! criptográfica — usa um gerador congruencial simples (determinístico,
//! reprodutível entre execuções).

use crate::graph::Graph;
use crate::models::{EdgeKind, NodeKind};
use crate::store::Catalog;
use crate::models::Product;

struct SimpleRng(u64);

impl SimpleRng {
    fn new(seed: u64) -> Self {
        SimpleRng(seed)
    }
    fn next(&mut self) -> u64 {
        // Gerador congruencial linear (parâmetros de Numerical Recipes).
        self.0 = self.0.wrapping_mul(6364136223846793005).wrapping_add(1);
        self.0
    }
    fn range(&mut self, bound: u32) -> u32 {
        (self.next() % bound as u64) as u32
    }
}

/// Constrói um grafo + catálogo sintéticos com `n_clients` clientes,
/// `n_products` produtos e `n_categories` categorias, com `edges_per_client`
/// compras/avaliações aleatórias por cliente e `similarity_per_product`
/// arestas de similaridade por produto.
pub fn build_synthetic(
    n_clients: u32,
    n_products: u32,
    n_categories: u32,
    edges_per_client: u32,
```

```
    similarity_per_product: u32,
    seed: u64,
) -> (Graph, Catalog) {
    let mut rng = SimpleRng::new(seed);
    let total_nodes = (n_clients + n_products + n_categories) as usize;
    let mut graph = Graph::with_capacity(total_nodes);
    let mut catalog = Catalog::with_capacity(n_products as usize);

    // ids: categorias [0, n_categories) | produtos [1000, 1000+n_products)
    // | clientes [10_000_000, 10_000_000+n_clients)
    const PRODUCT_OFFSET: u32 = 1_000;
    const CLIENT_OFFSET: u32 = 10_000_000;

    for c in 0..n_categories {
        graph.add_node(c, NodeKind::Category, &format!("Categoria {c}"));
    }

    for p in 0..n_products {
        let id = PRODUCT_OFFSET + p;
        let category_id = rng.range(n_categories.max(1));
        graph.add_node(id, NodeKind::Product, &format!("Produto {p}"));
        graph.add_edge(id, category_id, EdgeKind::BelongsTo, 1.0);
        catalog.register(Product {
            id,
            name: format!("Produto {p}"),
            category_id,
            price: 10.0 + (rng.range(1000) as f64) / 10.0,
        });
    }

    // Arestas de similaridade produto-produto.
    for p in 0..n_products {
        let id = PRODUCT_OFFSET + p;
        for _ in 0..similarity_per_product {
            let other = PRODUCT_OFFSET + rng.range(n_products.max(1));
            if other != id {
                let weight = 0.3 + (rng.range(70) as f64) / 100.0;
                graph.add_edge(id, other, EdgeKind::Similarity, weight);
            }
        }
    }

    for c in 0..n_clients {
        let id = CLIENT_OFFSET + c;
        graph.add_node(id, NodeKind::Client, &format!("Cliente {c}"));
        for _ in 0..edges_per_client {
            let product = PRODUCT_OFFSET + rng.range(n_products.max(1));
            let weight = 0.5 + (rng.range(50) as f64) / 100.0;
            graph.add_edge(id, product, EdgeKind::Purchase, weight);
        }
    }

    (graph, catalog)
}
```

src/main.rs

```
use megastore::dataset;
use megastore::graph;
use megastore::matrix;
use megastore::models;
use megastore::recommend;
use megastore::store;
use megastore::models::NodeKind;
use std::time::Instant;

fn main() {
    println!("=== ConectaStore — Sistema de Recomendação de Produtos Baseado em Grafos ===\
n");

    demo_pequena();
    println!();
```

```
    benchmark_volumes();
    println!();
    compare_matrix_vs_list();
}

/// Demonstração funcional com um grafo pequeno e legível, mostrando
/// cadastro, consulta e recomendação — usada nas capturas de tela do
/// relatório (Apêndice B).
fn demo_pequena() {
    let (mut graph, mut catalog) = (graph::Graph::new(), store::Catalog::new());

    graph.add_node(0, NodeKind::Category, "Eletrônicos");
    graph.add_node(1, NodeKind::Category, "Livros");

    graph.add_node(1000, NodeKind::Product, "Fone Bluetooth");
    graph.add_node(1001, NodeKind::Product, "Carregador USB-C");
    graph.add_node(1002, NodeKind::Product, "Livro de Rust");
    graph.add_node(1003, NodeKind::Product, "Mousepad");

    catalog.register(models::Product {
        id: 1000,
        name: "Fone Bluetooth".into(),
        category_id: 0,
        price: 199.90,
    });
    catalog.register(models::Product {
        id: 1001,
        name: "Carregador USB-C".into(),
        category_id: 0,
        price: 49.90,
    });
    catalog.register(models::Product {
        id: 1002,
        name: "Livro de Rust".into(),
        category_id: 1,
        price: 89.90,
    });
    catalog.register(models::Product {
        id: 1003,
        name: "Mousepad".into(),
        category_id: 0,
        price: 29.90,
    });

    graph.add_edge(1000, 0, models::EdgeKind::BelongsTo, 1.0);
    graph.add_edge(1001, 0, models::EdgeKind::BelongsTo, 1.0);
    graph.add_edge(1002, 1, models::EdgeKind::BelongsTo, 1.0);
    graph.add_edge(1003, 0, models::EdgeKind::BelongsTo, 1.0);

    graph.add_edge(1000, 1001, models::EdgeKind::Similarity, 0.8);
    graph.add_edge(1000, 1003, models::EdgeKind::Similarity, 0.6);
    graph.add_edge(1001, 1003, models::EdgeKind::Similarity, 0.5);

    graph.add_node(50, NodeKind::Client, "Cliente Eduardo");
    graph.add_edge(50, 1000, models::EdgeKind::Purchase, 1.0);
    graph.add_edge(50, 1002, models::EdgeKind::Rating, 0.9);

    println!("Catálogo: {} produtos cadastrados.", catalog.len());
    println!(
        "Consulta direta (id 1002): {:?}\n",
        catalog.get(1002).map(|p| &p.name)
    );

    let recs = recommend::recommend(&graph, 50, 2, 5);
    println!("Recomendações (BFS) para o Cliente Eduardo:");
    for r in &recs {
        let nome = catalog
            .get(r.product_id)
            .map(|p| p.name.as_str())
            .unwrap_or("?");
        println!(
            "  - {} (score={:.2}, distância={})",
            nome, r.score, r.distance
        );
    }
```

```
}

/// Mede o tempo de construção do grafo (já com HashMap pré-alocado via
/// with_capacity) e de geração de recomendações para diferentes volumes
/// de dados — dados usados na seção 6 do relatório (Desempenho e
/// Escalabilidade). Mais pontos de volume que a versão anterior, para
/// sustentar melhor a curva de crescimento.
fn benchmark_volumes() {
    let volumes = [
        (500u32, 2_500u32),
        (2_000, 10_000),
        (10_000, 50_000),
        (50_000, 250_000),
        (150_000, 750_000),
    ];

    println!("=== Benchmark: tempo de construção (com pré-alocação) e recomendação por volume
===");
    println!(
        "{:>12} | {:>12} | {:>10} | {:>14} | {:>16}",
        "clientes", "produtos", "arestas", "constr.(ms)", "recom. BFS(µs)"
    );

    for (n_clients, n_products) in volumes {
        let start_build = Instant::now();
        let (graph, _catalog) =
            dataset::build_synthetic(n_clients, n_products, 50, 5, 3, 42);
        let build_time = start_build.elapsed();

        let sample_client = 10_000_000; // primeiro cliente gerado
        let start_rec = Instant::now();
        let _recs = recommend::recommend(&graph, sample_client, 2, 10);
        let rec_time = start_rec.elapsed();

        println!(
            "{:>12} | {:>12} | {:>10} | {:>14.2} | {:>16.2}",
            n_clients,
            n_products,
            graph.edge_count(),
            build_time.as_secs_f64() * 1000.0,
            rec_time.as_secs_f64() * 1_000_000.0
        );
    }
}

/// Compara empiricamente lista de adjacência × matriz de adjacência em
/// escala pequena (a matriz não é viável em escala real — ver Seção 3.4).
/// Mede o espaço realmente ocupado por cada representação para o mesmo
/// conjunto de vértices e arestas.
fn compare_matrix_vs_list() {
    let volumes = [(20u32, 80u32), (100, 400), (300, 1_200), (600, 2_400)];

    println!("=== Comparação lista × matriz de adjacência (espaço ocupado) ===");
    println!(
        "{:>10} | {:>10} | {:>10} | {:>16} | {:>16} | {:>10}",
        "vértices", "arestas", "V²", "lista (bytes)", "matriz (bytes)", "razão"
    );

    for (n_clients, n_products) in volumes {
        let (graph, _catalog) = dataset::build_synthetic(n_clients, n_products, 10, 4, 2, 7);

        let node_ids: Vec<models::NodeId> = (0..10)
            .chain(1_000..1_000 + n_products)
            .chain(10_000_000..10_000_000 + n_clients)
            .collect();
        let mut mat = matrix::AdjacencyMatrix::new(&node_ids);
        for &id in &node_ids {
            for edge in graph.neighbors(id) {
                mat.add_edge(id, edge.target, edge.weight);
            }
        }

        let v = graph.node_count();
        let e = graph.edge_count();
        // Estimativa conservadora do custo da lista: um ponteiro/offset por
```

```
        // vértice (8 bytes) + estrutura Edge por aresta (target u32 + kind
        // enum + weight f64 ≈ 24 bytes com alinhamento).
        let list_bytes = v * 8 + e * 24;
        let matrix_bytes = mat.bytes_used();

        println!(
            "{:>10} | {:>10} | {:>10} | {:>16} | {:>16} | {:>9.1}x",
            v,
            e,
            v * v,
            list_bytes,
            matrix_bytes,
            matrix_bytes as f64 / list_bytes as f64
        );
    }
}
```

tests/integration_test.rs

```
//! Testes de integração: validam o fluxo completo — cadastro de produtos,
//! construção do grafo e geração de recomendações — como um usuário real
//! do sistema utilizaria.

use megastore::dataset::build_synthetic;
use megastore::graph::Graph;
use megastore::models::{EdgeKind, NodeKind, Product};
use megastore::recommend::recommend;
use megastore::store::Catalog;

#[test]
fn fluxo_completo_cadastro_grafo_recomendacao() {
    let mut graph = Graph::new();
    let mut catalog = Catalog::new();

    graph.add_node(1, NodeKind::Client, "Cliente 1");
    graph.add_node(100, NodeKind::Product, "Teclado Mecânico");
    graph.add_node(101, NodeKind::Product, "Mouse Gamer");

    catalog.register(Product {
        id: 100,
        name: "Teclado Mecânico".into(),
        category_id: 0,
        price: 350.0,
    });
    catalog.register(Product {
        id: 101,
        name: "Mouse Gamer".into(),
        category_id: 0,
        price: 180.0,
    });

    graph.add_edge(1, 100, EdgeKind::Purchase, 1.0);
    graph.add_edge(100, 101, EdgeKind::Similarity, 0.9);

    let recs = recommend(&graph, 1, 2, 5);
    assert_eq!(recs.len(), 1);
    assert_eq!(recs[0].product_id, 101);
    assert!(catalog.get(101).is_some());
}

#[test]
fn dataset_sintetico_gera_volume_esperado_de_vertices() {
    let (graph, catalog) = build_synthetic(50, 200, 10, 4, 2, 7);

    // clientes + produtos + categorias
    assert_eq!(graph.node_count(), 50 + 200 + 10);
    assert_eq!(catalog.len(), 200);
    assert!(graph.edge_count() > 0);
}

#[test]
fn recomendacao_em_grafo_grande_nao_gera_duplicados() {
```

```
    let (graph, _catalog) = build_synthetic(200, 500, 20, 6, 3, 99);
    let recs = recommend(&graph, 10_000_000, 2, 20);

    let mut ids: Vec<_> = recs.iter().map(|r| r.product_id).collect();
    let before = ids.len();
    ids.sort();
    ids.dedup();
    assert_eq!(before, ids.len());
}
```

rodar_testes.sh

```
#!/usr/bin/env bash
# Script de testes do ConectaStore — gera, em ordem, tudo que precisa
# ser capturado em print para o Apêndice B do relatório.
#
# Como usar (Pop!_OS / qualquer Linux com Rust instalado):
#   1. Extraia o ConectaStore_Projeto_Rust.zip
#   2. Copie este arquivo para dentro da pasta "megastore" (mesmo nível do Cargo.toml)
#   3. No terminal, dentro da pasta megastore:
#        chmod +x rodar_testes.sh
#        ./rodar_testes.sh
#
# O script pausa entre cada etapa para você tirar o print com calma.
# Toda a saída também é salva em resultados_conectastore.txt, caso
# prefira me colar o texto em vez de (ou além de) enviar prints.

set -e

if [ ! -f "Cargo.toml" ]; then
  echo "ERRO: rode este script de dentro da pasta 'megastore' (onde está o Cargo.toml)."
  exit 1
fi

echo "======================================================================"
echo " ConectaStore — script de testes para o relatório"
echo "======================================================================"
echo

echo ">>> Verificando o Rust instalado:"
cargo --version
rustc --version
echo

read -p "Pressione ENTER para compilar em modo release... " _
echo
cargo build --release
echo
echo "###  PRINT 1: capture a saída acima (deve terminar em 'Finished release [optimized]')."
echo
read -p "Pressione ENTER para rodar os testes automatizados... " _
echo
cargo test --release
echo
echo "###  PRINT 2: capture a linha 'test result: ok. 14 passed; 0 failed'."
echo
read -p "Pressione ENTER para rodar a demonstração e os benchmarks (saída mais longa)... " _
echo

# Salva a saída completa em arquivo e também mostra na tela.
cargo run --release 2>&1 | tee resultados_conectastore.txt

echo
echo "======================================================================"
echo "Rolando o terminal para cima, capture três prints separados da saída acima:"
echo "###  PRINT 3: catálogo cadastrado + consulta direta + recomendações do Cliente Eduardo"
echo "###  PRINT 4: tabela 'Benchmark: tempo de construção (...) e recomendação por volume'
(5 linhas)"
echo "###  PRINT 5: tabela 'Comparação lista × matriz de adjacência (espaço ocupado)' (4
linhas)"
echo "======================================================================"
echo
```

```
echo "A saída completa também foi salva em: resultados_conectastore.txt"
echo "Se preferir, me envie o conteúdo desse arquivo em vez dos prints 4 e 5 —"
echo "eu extraio os números de lá para atualizar as Tabelas 1 e 2 do relatório."
```

README.md

````
# ConectaStore — Sistema de Recomendação de Produtos Baseado em Grafos

## Objetivo e funcionamento
Sistema de recomendação para a MegaStore que modela produtos, clientes e
categorias como vértices de um grafo dirigido e ponderado, e compras,
avaliações, similaridades e pertencimento a categoria como arestas. As
recomendações são geradas por busca em largura (BFS) a partir de um
cliente ou produto, priorizando itens mais próximos no grafo e evitando
duplicatas.

## Tecnologias e estruturas utilizadas
- **Linguagem**: Rust (edition 2021)
- **Grafo**: lista de adjacência (`HashMap<NodeId, Vec<Edge>>`), pré-alocada
  via `with_capacity` para reduzir custo de rehashing na construção
- **Matriz de adjacência**: implementada em `src/matrix.rs` apenas para
  comparação empírica de espaço com a lista (não é usada em produção —
  ver discussão no relatório, Seção 3.4)
- **Catálogo de produtos**: `HashMap<NodeId, Product>` para acesso O(1)
- **Recomendação**: BFS com `VecDeque` (fila) e `HashSet` (visitados e
  prevenção de duplicados); variante DFS incluída para comparação
- **Geração de dados sintéticos**: gerador congruencial linear próprio,
  sem dependências externas

## Instruções para compilação e execução
```bash
cargo build --release
./target/release/megastore
```
Alternativamente, o script `rodar_testes.sh` (na raiz do projeto) executa
compilação, testes e a demonstração/benchmarks em sequência, com pausas
para captura de prints:
```bash
./rodar_testes.sh
```

## Instruções para execução dos testes
```bash
cargo test --release
```
14 testes (11 unitários dos módulos `graph`, `recommend`, `store` e
`matrix`, mais 3 de integração), todos cobrindo cadastro, construção do
grafo, geração de recomendação e prevenção de duplicados.

## Exemplos de uso
A execução de `cargo run --release` roda automaticamente:
1. Uma demonstração pequena com 4 produtos e 1 cliente, mostrando cadastro,
   consulta direta e recomendação.
2. Um benchmark de desempenho com cinco volumes de dados (500 a 150 mil
   clientes), já com as estruturas pré-alocadas via `with_capacity`.
3. Uma comparação empírica de espaço entre lista e matriz de adjacência
   em pequena escala (até 3 mil vértices), demonstrando o crescimento
   quadrático da matriz.

## Arquitetura da solução
```
megastore/
├── src/
│   ├── main.rs        (demonstração + benchmarks)
│   ├── lib.rs          (exposição dos módulos para testes)
│   ├── models.rs       (Node, Edge, Product)
│   ├── graph.rs        (grafo em lista de adjacência, pré-alocável)
│   ├── matrix.rs        (matriz de adjacência — só para comparação)
│   ├── recommend.rs    (algoritmos BFS e DFS de recomendação)
│   ├── store.rs        (catálogo de produtos, pré-alocável)
│   └── dataset.rs       (gerador de dados sintéticos)
├── tests/
````
````

```
│   └── integration_test.rs
├── Cargo.toml
├── rodar_testes.sh    (script de testes/prints para o relatório)
└── README.md
```

Resultados dos testes de desempenho

Clientes	Produtos	Arestas	Construção (ms)	Recomendação BFS (µs)
500	2.500	12.499	1,73	8,57
2.000	10.000	49.999	6,52	12,21
10.000	50.000	249.998	39,02	10,88
50.000	250.000	1.249.999	207,35	11,35
150.000	750.000	3.749.997	869,35	10,41

Comparação lista × matriz de adjacência (espaço ocupado)

Vértices	Arestas	V²	Lista (bytes)	Matriz (bytes)	Razão
110	316	12.100	8.464	96.800	11,4×
510	1.596	260.100	42.384	2.080.800	49,1×
1.510	4.796	2.280.100	127.184	18.240.800	143,4×
3.010	9.597	9.060.100	254.408	72.480.800	284,9×

Máquina de referência: notebook do autor (Samsung Expert, 2018, Intel Core i7-8550U 1,80 GHz, 11 GiB de RAM, SSD de 428 GB, Pop!_OS), build `--release`, otimização O3, execução via `rodar_testes.sh`. Discussão completa no relatório técnico, Seções 3.4, 5.3 e 6.

Link do vídeo pitch

A preencher após a gravação (até 4 minutos), publicado no YouTube como não listado ou em plataforma equivalente.

APÊNDICE B — PRINTS DE EXECUÇÃO E TESTES

Capturas de tela da execução do ConectaStore no notebook do autor (Samsung Expert, 2018, Pop!_OS), via VS Code, comprovando a compilação, a bateria de testes automatizados, os resultados funcionais e de desempenho relatados nas Seções 6.2 e 3.4/5.3 e as especificações da máquina utilizada. Numeração de figuras deste apêndice segue prefixo B, conforme ABNT NBR 14724.

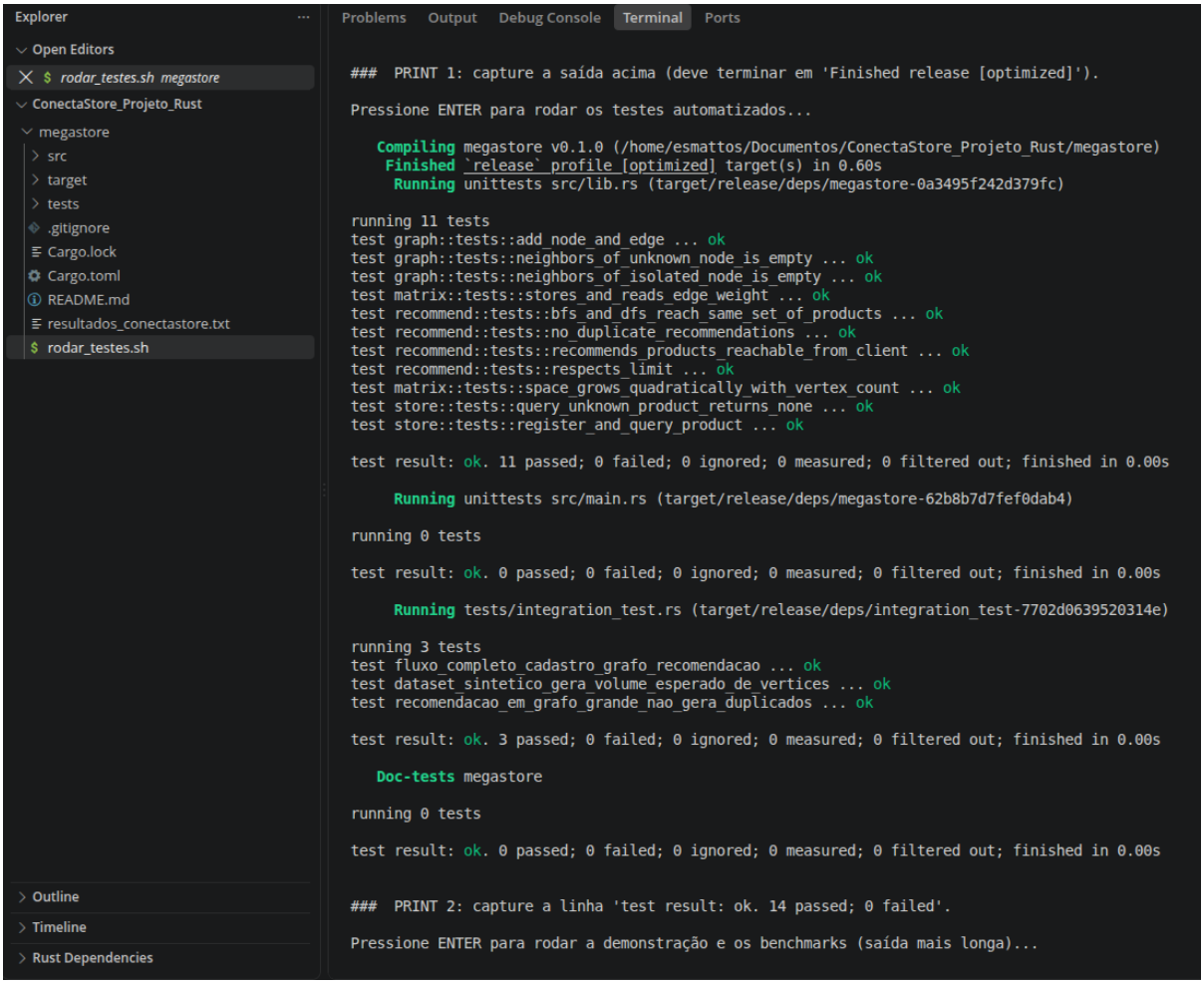


Figura B.1 — Compilação em modo release e execução dos 14 testes automatizados (11 unitários + 3 de integração), todos com resultado ok. Fonte: captura de tela do autor, terminal integrado do VS Code

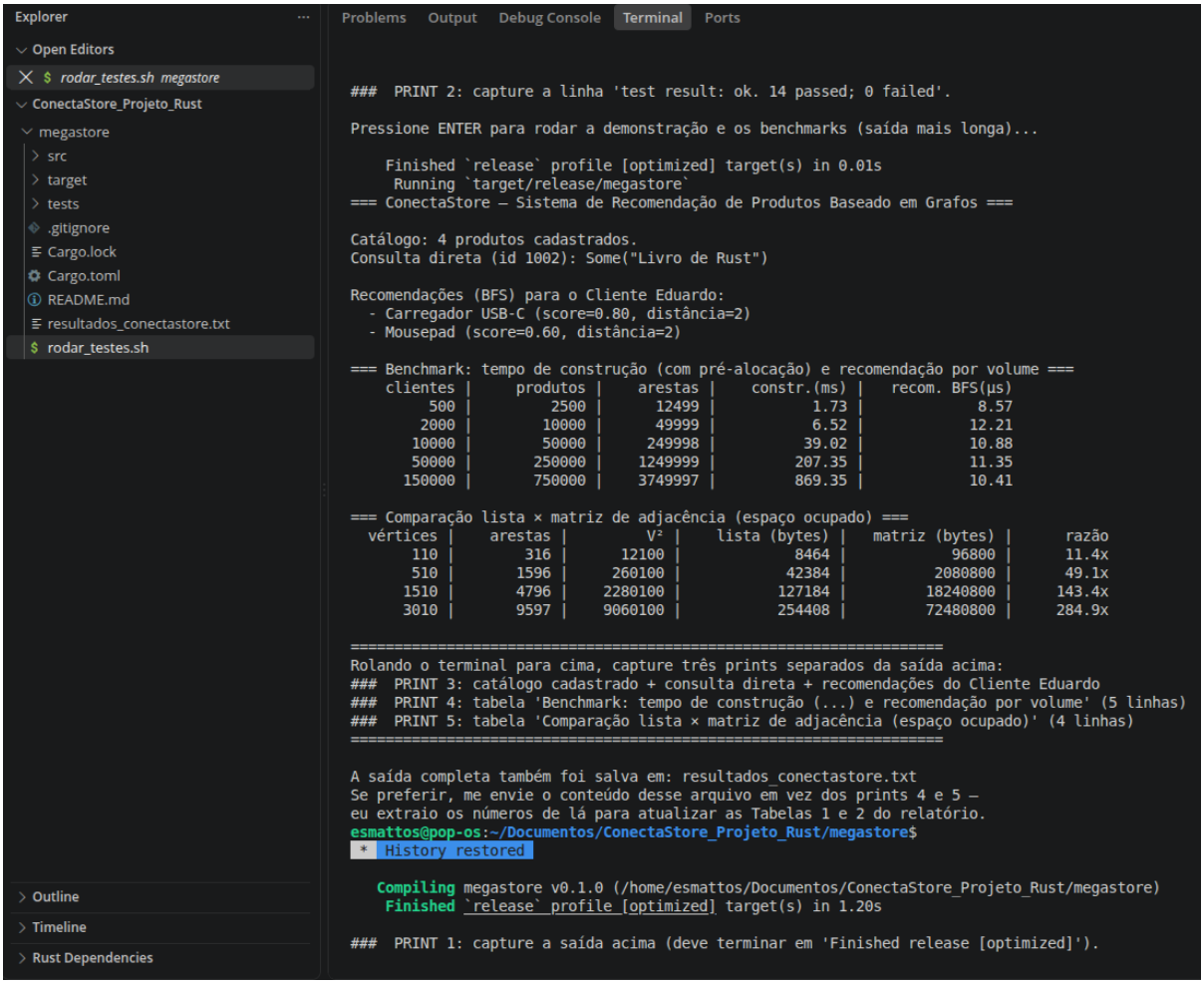


Figura B.2 — Demonstração funcional (catálogo, consulta e recomendações), tabela de benchmark por volume de dados e tabela de comparação lista × matriz de adjacência. Fonte: captura de tela do autor, terminal integrado do VS Code

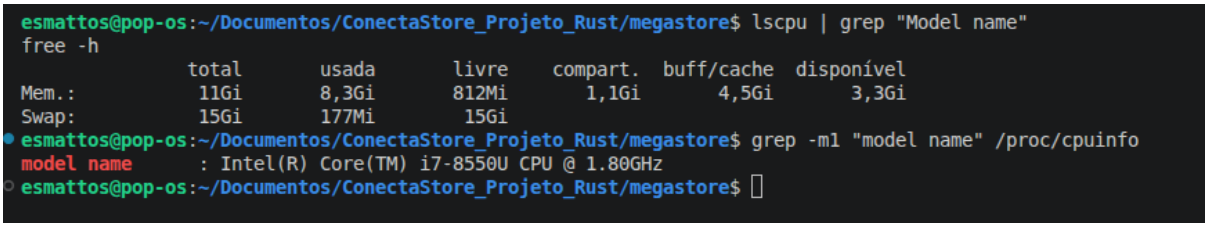


Figura B.3 — Especificações da máquina utilizada nos testes: processador Intel Core i7-8550U (1,80 GHz) e memória RAM de 11 GiB. Fonte: captura de tela do autor, terminal integrado do VS Code